

```

module_param(debug, bool, 0644);

#define DEBUG_ENT(fmt, arg...) do { if (debug) \
    printk(KERN_DEBUG "random %04d %04d %04d: " \
    fmt, \
    input_pool.entropy_count, \
    blocking_pool.entropy_count, \
    nonblocking_pool.entropy_count, \
    ## arg); } while (0)

#else
#define DEBUG_ENT(fmt, arg...) do {} while (0)
#endif

struct entropy_store;
struct entropy_store {
    /* mostly-read data: */
    struct poolinfo *poolinfo;
    __u32 *pool;
    const char *name;
    int limit;
    struct entropy_store *pull;

    /* read-write data: */
    spinlock_t lock;
    __cacheline_aligned_in_smp;
    unsigned add_ptr;
    int entropy_count;
    int input_rotate;
};

```

It will be good if you could get a copy of *A Mathematical Theory of Communication* (written by Shannon himself). In the book, the idea of information theory is discussed and Shannon entropy is introduced from scratch.

Shannon entropy is an important concept when we deal with random number generators. Roughly, I can say that high entropy corresponds to 'less useful information', which in turn corresponds to a large amount of random stuff, in a set of characters. Linux has an entropy pool, which has data from non-deterministic device events, making it purely random. It also calculates the 'entropy level change' (technically called *entropy estimate*) when the data is fed into the pool (and is used as a measure of the uncertainty). It does the same when there is a reduction in randomness. `add_timer_randomness` is shown below:

```

static void add_timer_randomness(struct timer_rand_state *state,
unsigned num)
{
    struct {
        cycles_t cycles;
        long jiffies;
        unsigned num;
    } sample;

```

```

    long delta, delta2, delta3;

    extern void add_input_randomness(unsigned int type, unsigned int code,
        unsigned int value)
    {
        static unsigned char last_value;

        /* ignore autorepeat and the like */
        if (value == last_value)
            return;

        DEBUG_ENT("input event\n");
        last_value = value;
        add_timer_randomness(&input_timer_state,
            (type << 4) ^ code ^ (code >> 4) ^ value);
    }

    void add_interrupt_randomness(int irq)
    {
        if (irq >= NR_IRQS || irq_timer_state[irq] == 0)
            return;

        DEBUG_ENT("irq event %d\n", irq);
        add_timer_randomness(irq_timer_state[irq], 0x100 + irq);
    }

    void add_disk_randomness(struct gendisk *disk)
    {
        if (!disk || !disk->random)
            return;

        /* first major is 1, so we get >= 0x200 here */
        DEBUG_ENT("disk event %d %d\n", disk->major, disk->first_minor);

        add_timer_randomness(disk->random,
            0x100 + MKDEV(disk->major, disk->first_minor));
    }

EXPORT_SYMBOL(add_disk_randomness);

```

Linux 1.3.30 saw the introduction of kernel random number generator, which is considered as a useful tool by programmers.

With that we have reached the end of today's voyage. Our next destination point will be time management, and later we will move on to kernel synchronisation. Wait till the next instalment to hack more kernel topics.

Happy Kernel Hacking! 

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com